

What is SnowSQL?

SnowSQL is a **Python-based Command Line Interface (CLI)** used to connect to **Snowflake** from operating systems like:

- Windows
- Linux
- macOS

It allows users to interact with Snowflake directly from the **terminal or command prompt**.

Using SnowSQL we can:

- Execute SQL queries
- Create database objects (tables, schemas, warehouses)
- Perform DDL & DML operations
- Load and unload data
- Run scripts for automation
- Manage Snowflake environments

In simple terms:

👉 **SnowSQL = Terminal tool to communicate with Snowflake.**

Ways to Access Snowflake

Snowflake can be accessed through multiple interfaces depending on the use case.

1 Snowflake Web Interface

Browser-based UI used for:

- Writing SQL queries
- Managing databases
- Monitoring warehouses
- Viewing query history

Snowflake recommends using **Google Chrome**.

2 SnowSQL (Command Line Client)

A terminal-based tool used mainly by:

- Data Engineers
- Developers
- Automation pipelines

Example usage:

```
snowsqli -a account_name -u username
```

3 JDBC / ODBC Drivers

Used by applications to connect to Snowflake.

Examples:

- Java applications → JDBC
 - BI tools → ODBC
-

4 Third-Party Tools

Many tools integrate directly with Snowflake.

Examples:

- Tableau
 - Matillion
 - Power BI
 - Informatica
 - Fivetran
-

Installing SnowSQL

SnowSQL can be downloaded from the **Snowflake website**.

The installer includes all required dependencies.

Supported Operating Systems

Windows

- Windows 7 or later
- Windows Server 2008 R2 or later

macOS

- Version 10.12 or later

Linux

- CentOS 6+
 - Ubuntu 14+
-

Installation Steps

- 1 Login to Snowflake Web UI
- 2 Navigate to
Help → Downloads
- 3 Download **SnowSQL CLI** for your OS.
- 4 Run the installer.
- 5 Follow the installation wizard.
- 6 After installation you will see

Installation is Complete

Now SnowSQL is ready to use.

Configuring SnowSQL

After installation, SnowSQL must be configured using the **config file**.

Configuration file name:

config

Location:

Linux / macOS

~/.snowsql/

Windows

C:\Users\<username>\.snowsql\

Important rule:

👉 The config file must be saved in **UTF-8 encoding**.

SnowSQL Configuration Sections

The configuration file has **three main sections**:

1 Connection Settings

Stores account login details.

2 Configuration Options

Stores CLI settings.

3 Variables

Stores reusable variables

Connection Settings

To connect SnowSQL to your Snowflake account, you need:

- Account Name
 - Username
 - Password
 - Region
-

Snowflake URL Format

AWS (US-West)

`https://<account_name>.snowflakecomputing.com`

AWS (Other Regions)

`https://<account_name>.<region>.snowflakecomputing.com`

Azure

`https://<account_name>.<region>.azure.snowflakecomputing.com`

Example:

`https://ru06473.ap-southeast-1.snowflakecomputing.com`

Testing SnowSQL Connection

Use the following command:

`snowsql -a account_name -u username`

Example

`snowsql -a ru06473.ap-southeast-1 -u SKILLSCALER`

If successful, SnowSQL will open an interactive session.

Saving Credentials in Config File

To avoid typing credentials every time, edit the config file.

Example configuration:

```
accountname = wr82310
username = SKILLSCALER1
password = xxxxxxxx
region = ap-south-1
```

Then simply run:

```
snowsql
```

 Important:

Passwords are stored in **plain text**, so the config file must be **secured properly**.

SnowSQL Variables

Variables allow you to store frequently used values like:

- database name
- schema name
- table name
- date values

This improves productivity.

Example variable definition:

```
[variables]
```

```
database_name = SNOWFLAKE_SAMPLE_DATA
schema_name = TPCH_SF001
table_name = NATION
```

Using Variables in SnowSQL

Start SnowSQL:

```
snowsql
```

Enable substitution:

```
!set variable_substitution=true
```

Use variables:

```
USE "&database_name";
USE SCHEMA "&schema_name";
```

```
SELECT COUNT(*) FROM "&table_name";
```

Command Line Variables

Variables can also be defined when starting SnowSQL.

Example:

```
snowsql -D tablename=NATION -s TPCH_SF001 -d SNOWFLAKE_SAMPLE_DATA
```

Then run:

```
!set variable_substitution=true  
select count(*) from "&tablename";
```

Variables in Active Session

Variables can also be defined inside an active session.

Example:

```
!define tablename=NATION  
!set variable_substitution=true  
SELECT COUNT(*) FROM "&tablename";
```

SnowSQL Commands

SnowSQL commands start with !

Example:

```
!help
```

This displays all available commands.

Common SnowSQL Commands

Abort Query

```
!abort <query_id>
```

Create Connection

```
!connect <connection_name>
```

Define Variable

```
!define variable=value
```

Open SQL Editor

!edit

Show Options

!options

Exit Session

!exit

Quit SnowSQL

!quit

Difference:

!exit → closes one connection

!quit → closes all connections

Execute SQL File

!source filename.sql

Save Query Output to File

!spool output.txt

Stop writing output

!spool off

Multiple Connections in SnowSQL

SnowSQL supports connecting to **multiple environments**.

Example environments:

- Development
- Testing
- Production

These can be configured in the config file.

Example:

```
[connections.development]
password=xxxx
warehousename=DEVELOPMENT
```

```
[connections.production]
password=xxxx
warehousename=PRODUCTION
```

Creating Warehouses for Environments

Example:

```
CREATE WAREHOUSE DEVELOPMENT
WAREHOUSE_SIZE='XSMALL'
AUTO_SUSPEND=600
AUTO_RESUME=TRUE;

CREATE WAREHOUSE PRODUCTION
WAREHOUSE_SIZE='XSMALL'
AUTO_SUSPEND=600
AUTO_RESUME=TRUE;
```

Switching Connections

```
!connect development
!connect production
```

Connection stack example:

Development → Production → Exit → Back to Development

Data Loading Using SnowSQL

SnowSQL can be used to **automate data pipelines** using the COPY command.

Example file:

zips2000.csv

Step 1 – Set Environment

```
USE WAREHOUSE COMPUTE_WH;
USE DATABASE DEMO_DB;
USE SCHEMA PUBLIC;
```

Step 2 – Create Table

```
CREATE TABLE ZIPCODES2000_SNOWSQL  
(  
  ZIPCODE STRING,  
  LON DOUBLE,  
  LAT DOUBLE  
);
```

Step 3 – Upload File to Stage

```
PUT file:///Users/path/zips2000.csv  
@DEMO_DB.PUBLIC.%zipcodes2000_snowsql;
```

Step 4 – Load Data

```
COPY INTO zipcodes2000_snowsql  
FROM @%zipcodes2000_snowsql  
FILE_FORMAT = (  
  TYPE = CSV  
  FIELD_OPTIONALLY_ENCLOSED_BY='\"'  
  SKIP_HEADER = 1  
);
```

Step 5 – Verify Data

```
SELECT * FROM zipcodes2000_snowsql;
```

SnowSQL Connection Command

Basic syntax:

```
snowsql -a accountname -u username  
-d database  
-s schema
```

Example:

```
snowsql -a xta99637.us-east-1  
-u vithaljs  
-d demo_db  
-s public
```

Common SnowSQL Command Options

Option	Description
-a	Account name
-u	Username
-d	Database
-s	Schema
-r	Role
-w	Warehouse
-q	Execute query
-f	Run SQL file
-D	Define variable
-c	Use connection
-v	Show version
--help	Show help

Real-Time Use Case

In real projects SnowSQL is used for:

- Automating data pipelines
- Scheduling ETL jobs
- Running batch scripts
- Loading large files into Snowflake
- Running CI/CD data workflows

Example:

```
snowsql -f load_data.sql
```

This runs the entire pipeline automatically.
